1a47ef6e-03ba-413c-8aee-1c03f870d3

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'OPCFoundation/UA-.NETStandard' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/UA-.NETStandard/UA-.NETStandard (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-19T17:30:32.739Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
[Server/Client] Support RSAEncryptedSecret format for UserNameIdentityToken with legacy .NET compatibility
</pr_title>

<pr_description>
Proposed changes

This PR now covers full SDK handling for RSA username tokens by adding both server-side **decryption** and client-side **encryption** support for `RsaEncryptedSecret` (UA 1.05), while preserving legacy RSA behavior for compatibility.

- **Server token format support**
  - `UserNameIdentityTokenHandler.Decrypt(...)` accepts RSA `RsaEncryptedSecret` payloads.

  - Legacy RSA `EncryptedData` decryption remains as fallback.
  - `RsaEncryptedSecret` DataType usage now pulls from global constants (`DataTypeIds.RsaEncryptedSecret`) instead of local numeric constants.

- **Client token format support**
  - `UserNameIdentityTokenHandler.Encrypt(...)` now applies RSAEncryptedSecret based on password size for RSA policies:
    - `password.Length > 64` bytes → serialize as `RsaEncryptedSecret`
    - `password.Length <= 64` bytes → keep legacy RSA `EncryptedData` format

- **Validation and cryptographic flow**
  - Validates ExtensionObject type/encoding, SecurityPolicy URI, certificate hash, signing timestamp window, signature, and nonce.
  - Decrypts `KeyData`, derives signing/encryption/IV material, decrypts payload, and securely clears sensitive intermediate buffers.

- **Legacy target compatibility follow-up**
  - Added local compatibility helpers in `UserNameIdentityTokenHandler` to avoid APIs unavailable on older targets:
    - Replaced direct `CryptographicOperations.ZeroMemory(...)` calls with a helper that uses:
      - `CryptographicOperations.ZeroMemory` on `NET8_0_OR_GREATER`
      - `Array.Clear` on earlier targets
    - Replaced `SHA1.HashData(...)` with a compatible helper based on `SHA1.Create().ComputeHash(...)`
  - This resolves the CodeQL/build failures seen on older target frameworks (e.g., `net48`, `net472`, `netstandard2.1`) without changing protocol behavior.

- **Focused test coverage**
  - Added and extended `UserNameIdentityTokenHandlerTests` to cover:
    - RSAEncryptedSecret decrypt path
    - Legacy RSA decrypt compatibility
    - Encrypt behavior for short, threshold (64), and long passwords
    - Encrypt/decrypt roundtrip validation for both legacy and RSAEncryptedSecret paths

Related Issues

Handled by issue automation.

Types of changes

What types of changes does your code introduce?
Put an `x` in the boxes that apply. You can also fill these out after creating the PR.

- [x] Bugfix (non-breaking change which fixes an issue)
- [x] Enhancement (non-breaking change which adds functionality)
- [x] Test enhancement (non-breaking change to increase test coverage)
- [ ] Breaking change (fix or feature that would cause existing functionality to not work as expected, requires version increase of Nuget packages)
- [ ] Documentation Update (if none of the other choices apply)

Checklist

Put an `x` in the boxes that apply. You can also fill these out after creating the PR. If you're unsure about any of them, don't hesitate to ask. We're here to help! This is simply a reminder of what we are going to look for before merging your code.

- x] I have read the [CONTRIBUTING doc.
- x] I have signed the [CLA.
- [x] I ran tests locally with my changes, all passed.
- [x] I fixed all failing tests in the CI pipelines.
- [x] I fixed all introduced issues with CodeQL and LGTM.
- [x] I have added tests that prove my fix is effective or that my feature works and increased code coverage.
- [ ] I have added necessary documentation (if appropriate).
- [ ] Any dependent changes have been merged and published in downstream modules.

Further comments

The change remains backward-compatible by keeping legacy RSA `EncryptedData` for passwords up to 64 bytes and only switching to RSAEncryptedSecret for longer passwords, matching UA 1.05 guidance and enabling end-to-end interoperability between client and server paths. The compatibility helpers only adjust API

usage across target frameworks and do not alter wire format or cryptographic validation behavior.
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

</pr_comments>

<pr_reviews>

<comment_thread_resolved>
<file>Stack/Opc.Ua.Core/Stack/Types/UserNameIdentityTokenHandler.cs:43</file>
<original_commit>d37050f</original_commit>

</comment_thread_resolved>

</pr_reviews>

<pr_comments>

comment_old>

<comment_old>
<author>@copilot</author>

> @copilot for full sdk Support the Client should also apply this format aboth a cetrain PW length in the encrypt path
>

Implemented in 8561c93: client `Encrypt` now uses RSAEncryptedSecret for RSA policies when password length is >64 bytes, and keeps legacy RSA format for <=64 bytes. Added focused tests for short, threshold, and long password behavior with decrypt roundtrip.

</comment_old>

<comment_old>
<author>@marcschier</author>
@copilot Add a polyfill for CryptographicOperations. ZeroMemory which is not available in earlier versions of .net. see failing codeql pipeline.
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot Add a polyfill for CryptographicOperations. ZeroMemory which is not available in earlier versions of .net. see failing codeql pipeline.

Implemented in 4a559f3. I added a compatibility polyfill in `UserNameIdentityTokenHandler` for buffer zeroing (`ZeroMemory`) so older targets avoid `CryptographicOperations.ZeroMemory`, and also replaced `SHA1.HashData` with a compatible helper to fix the same failing CodeQL build matrix. No UI changes (no screenshot applicable).
</comment_old>

<comment_new>
<comment_id>4276437094</comment_id>
<author>@marcschier</author>
@copilot - the new tests do not compile for older .net SDK, see https://github.com/OPCFoundation/UA-.NETStandard/actions/runs/24633036015/job/72024739764?pr=3678. Add polyfills to ensure proper compilation against all target frameworks.
</comment_new>

</pr_comments>

</comments>

The last **7** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement><rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:

- If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.

<prev_assistant>

Addressed the new comment `4276437094` with minimal, targeted test compatibility fixes.

✅ What I changed
In `/home/runner/work/UA-.NETStandard/UA-.NETStandard/Tests/Opc.Ua.Core.Tests/Stack/Types/UserNameIdentityTokenHandlerTests.cs`:

- Replaced `RandomNumberGenerator.GetBytes(int)` usages with a local helper:
  - `GetRandomBytes(int)` → uses `RandomNumberGenerator.Create().GetBytes(byte[])`
- Replaced `SHA1.HashData(...)` with a local helper:
  - `ComputeSha1Hash(byte[])` → uses `SHA1.Create().ComputeHash(...)`
- Minor follow-up: renamed local `rng` variable to `randomNumberGenerator`.

✅ Validation performed
- Investigated CI failure using GitHub Actions MCP logs:
  - Run `24633036015`, job `72024739764`
- Verified local builds:
  - `dotnet build ...Opc.Ua.Core.Tests.csproj -f net48` ✅

  - `dotnet build ...Opc.Ua.Core.Tests.csproj -f net472` ✅
- Verified targeted tests:
  - `dotnet test ...Opc.Ua.Core.Tests.csproj -f net10.0 --filter FullyQualifiedName~UserNameIdentityTokenHandlerTests` ✅ (5/5)

✅ Commits
- `ebeb6ef` — test polyfills for legacy framework compatibility
- `0303570` — readability rename from review feedback

✅ Comment reply
Replied to `4276437094` with commit hash `0303570`.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'OPCFoundation/UA-.NETStandard' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/UA-.NETStandard/UA-.NETStandard (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-20T13:23:00.694+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
[Server/Client] Support RSAEncryptedSecret format for UserNameIdentityToken with legacy .NET and test compatibility
</pr_title>

<pr_description>
Proposed changes

This PR now covers full SDK handling for RSA username tokens by

```
adding both server-side **decryption** and client-side
**encryption** support for `RsaEncryptedSecret` (UA 1.05), while
preserving legacy RSA behavior for compatibility.

- **Server token format support**
  - `UserNameIdentityTokenHandler.Decrypt(...)` accepts RSA
`RsaEncryptedSecret` payloads.
  - Legacy RSA `EncryptedData` decryption remains as fallback.
  - `RsaEncryptedSecret` DataType usage now pulls from global
constants (`DataTypeIds.RsaEncryptedSecret`) instead of local
numeric constants.

- **Client token format support**
  - `UserNameIdentityTokenHandler.Encrypt(...)` now applies
RSAEncryptedSecret based on password size for RSA policies:
    - `password.Length > 64` bytes → serialize as
`RsaEncryptedSecret`
    - `password.Length <= 64` bytes → keep legacy RSA
`EncryptedData` format

- **Validation and cryptographic flow**
  - Validates ExtensionObject type/encoding, SecurityPolicy URI,
certificate hash, signing timestamp window, signature, and
nonce.
  - Decrypts `KeyData`, derives signing/encryption/IV material,
decrypts payload, and securely clears sensitive intermediate
buffers.

- **Legacy target compatibility follow-up (product code)**
  - Added local compatibility helpers in
`UserNameIdentityTokenHandler` to avoid APIs unavailable on
older targets:
    - Replaced direct `CryptographicOperations.ZeroMemory(...)`
calls with a helper that uses:
      - `CryptographicOperations.ZeroMemory` on
`NET8_0_OR_GREATER`
      - `Array.Clear` on earlier targets
    - Replaced `SHA1.HashData(...)` with a compatible helper
based on `SHA1.Create().ComputeHash(...)`
  - This resolves CodeQL/build failures seen on older target
frameworks (e.g., `net48`, `net472`, `netstandard2.1`) without
changing protocol behavior.

- **Legacy target compatibility follow-up (tests)**
  - Updated `UserNameIdentityTokenHandlerTests` for older
framework compilation:
    - Replaced `RandomNumberGenerator.GetBytes(int)` with a
```

```
local helper using
`RandomNumberGenerator.Create().GetBytes(byte[])`
    - Replaced `SHA1.HashData(...)` with a local helper using
`SHA1.Create().ComputeHash(...)`
  - This resolves test-project compilation failures on older
target frameworks in CI/CodeQL.

- **Focused test coverage**
  - Added and extended `UserNameIdentityTokenHandlerTests` to
cover:
    - RSAEncryptedSecret decrypt path
    - Legacy RSA decrypt compatibility
    - Encrypt behavior for short, threshold (64), and long
passwords
    - Encrypt/decrypt roundtrip validation for both legacy and
RSAEncryptedSecret paths

## Related Issues

Handled by issue automation.

## Types of changes

What types of changes does your code introduce?
_Put an `x` in the boxes that apply. You can also fill these out
after creating the PR._

- [x] Bugfix (non-breaking change which fixes an issue)
- [x] Enhancement (non-breaking change which adds functionality)
- [x] Test enhancement (non-breaking change to increase test
coverage)
- [ ] Breaking change (fix or feature that would cause existing
functionality to not work as expected, requires version increase
of Nuget packages)
- [ ] Documentation Update (if none of the other choices apply)

## Checklist

_Put an `x` in the boxes that apply. You can also fill these out
after creating the PR. If you're unsure about any of them, don't
hesitate to ask. We're here to help! This is simply a reminder
of what we are going to look for before merging your code._

- <a href="https://github.com/OPCFoundation/UA-.NETStandard/
blob/master/CONTRIBUTING.md">x] I have read the [CONTRIBUTING</
a> doc.
- <a href="https://opcfoundation.org/license/cla/
```

```
ContributorLicenseAgreementv1.0.pdf">x] I have signed the [CLA</
a>.
- [x] I ran tests locally with my changes, all passed.
- [x] I fixed all failing tests in the CI pipelines.
- [x] I fixed all introduced issues with CodeQL and LGTM.
- [x] I have added tests that prove my fix is effective or that
my feature works and increased code coverage.
- [ ] I have added necessary documentation (if appropriate).
- [ ] Any dependent changes have been merged and published in
downstream modules.

## Further comments

The change remains backward-compatible by keeping legacy RSA
`EncryptedData` for passwords up to 64 bytes and only switching
to RSAEncryptedSecret for longer passwords, matching UA 1.05
guidance and enabling end-to-end interoperability between client
and server paths. The compatibility helpers in both product code
and tests only adjust API usage across target frameworks and do
not alter wire format or cryptographic validation behavior.
</pr_description>

## Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@romanett</author>
@marcschier for full sdk Support the Client should also apply
this format aboth a cetrain PW length in the encrypt path
</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread_resolved>
<file>Stack/Opc.Ua.Core/Stack/Types/
UserNameIdentityTokenHandler.cs:43</file>
<original_commit>d37050f</original_commit>

<comment_old>
<author>@romanett</author>
Pull from global consts
</comment_old>
```

<comment_old>
<author>@marcschier</author>
@copilot, change const for 17545 to use the generated const in the generated Data types class.
</comment_old>

<comment_old>
<author>@copilot</author>
Updated in 8561c93: switched RSAEncryptedSecret type checks/encoding to use global constants (`DataTypeIds.RsaEncryptedSecret`) instead of a local numeric constant.
</comment_old>

</comment_thread_resolved>

</pr_reviews>

<pr_comments>

<comment_old>
<author>@marcschier</author>
@copilot for full sdk Support the Client should also apply this format aboth a cetrain PW length in the encrypt path

</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot for full sdk Support the Client should also apply this format aboth a cetrain PW length in the encrypt path
>

Implemented in 8561c93: client `Encrypt` now uses RSAEncryptedSecret for RSA policies when password length is >64 bytes, and keeps legacy RSA format for <=64 bytes. Added focused tests for short, threshold, and long password behavior with decrypt roundtrip.
</comment_old>

<comment_old>
<author>@marcschier</author>
@copilot Add a polyfill for CryptographicOperations. ZeroMemory which is not available in earlier versions of .net. see failing codeql pipeline.

</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot Add a polyfill for CryptographicOperations.ZeroMemory which is not available in earlier versions of .net. see failing codeql pipeline.

Implemented in 4a559f3. I added a compatibility polyfill in `UserNameIdentityTokenHandler` for buffer zeroing (`ZeroMemory`) so older targets avoid `CryptographicOperations.ZeroMemory`, and also replaced `SHA1.HashData` with a compatible helper to fix the same failing CodeQL build matrix. No UI changes (no screenshot applicable).
</comment_old>

<comment_old>
<author>@marcschier</author>
@copilot - the new tests do not compile for older .net SDK, see https://github.com/OPCFoundation/UA-.NETStandard/actions/runs/24633036015/job/72024739764?pr=3678. Add polyfills to ensure proper compilation against all target frameworks.
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot - the new tests do not compile for older .net SDK, see https://github.com/OPCFoundation/UA-.NETStandard/actions/runs/24633036015/job/72024739764?pr=3678. Add polyfills to ensure proper compi...

Implemented in 0303570. I added test-side compatibility polyfills in `UserNameIdentityTokenHandlerTests` by replacing `RandomNumberGenerator.GetBytes(int)` and `SHA1.HashData(...)` with helper methods (`RandomNumberGenerator.Create().GetBytes(byte[])` and `SHA1.Create().ComputeHash(...)`) so the tests compile on older target frameworks (net48/net472). No UI changes (no screenshot applicable).
</comment_old>

<comment_new>
<comment_id>4281114028</comment_id>
<author>@romanett</author>
@copilot Please extend the existing EnrcyptedSecret Class instead of implementing the serialization / deserialization

```
  in the UserNameIdentityToken.
  </comment_new>

</pr_comments>

</comments>

----
The last **9** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</
problem_statement><ModelSourceGeneratorGenerateNodeManager>true<
/ModelSourceGeneratorGenerateNodeManager>` MSBuild property; off
by default.
- Citations: Tools/Opc.Ua.SourceGeneration/
OPCFoundation.Opc.Ua.SourceGeneration.props:11; Applications/
ConsoleBoilerServer/ConsoleBoilerServer.csproj; Docs/
SourceGeneratedNodeManagers.md

**AOT logging**
- Fact: For NativeAOT OPC UA servers use
`DefaultTelemetry.Create(b => b.AddConsole())` from
Microsoft.Extensions.Logging.Console; do NOT use Serilog (AOT
compatibility varies).
- Citations: Applications/ConsoleBoilerServer/Program.cs:79;
Docs/SourceGeneratedNodeManagers.md; User input: "fully
NativeAOT therefore not use Serilog, but
Microsoft.Extensions.Logging.Console"

**source generator**
- Fact: Boolean fields in source-generated data types default to
false. The previous dataTypeQuirk that inverted boolean defaults
to true was removed.
- Citations: Tools/Opc.Ua.SourceGeneration.Core/Schema/
ModelDesignExtensions.cs:852, Tools/
Opc.Ua.SourceGeneration.Core/Generators/
DataTypeGenerator.cs:1156

**node id checks**
- Fact: Use `nodeId.IsNull` (instance property). The static
`NodeId.IsNull(nodeId)` and the `IsNullNodeId` extension are
both `[Obsolete]` and break TreatWarningsAsErrors builds.
- Citations: D:
\git\UA-.NETStandard\Libraries\Opc.Ua.Server\NodeManager\Fluent\
NodeManagerBuilder.cs:299; build error CS0618
NodeIdObsolete.IsNullNodeId / NodeIdStaticObsolete.IsNull

**source generators**
```

- Fact: New `ObjectMethodProxyGenerator` (in Opc.Ua.SourceGeneration.Core) emits typed async client wrappers for ObjectTypes with declared methods; gated by `GeneratorOptions.GenerateObjectMethodProxies` (off by default), MSBuild prop `ModelSourceGeneratorGenerateObjectMethodProxies`. Output namespace overridable via `ObjectMethodProxyNamespace`.
- Citations: D:\git\UA-.NETStandard3\Tools\Opc.Ua.SourceGeneration.Core\Generators\ObjectMethodProxyGenerator.cs, D:\git\UA-.NETStandard3\Tools\Opc.Ua.SourceGeneration.Core\GeneratorOptions.cs, D:\git\UA-.NETStandard3\Tools\Opc.Ua.SourceGeneration\OPCFoundation.Opc.Ua.SourceGeneration.props

test compatibility
- Fact: In Opc.Ua.Core.Tests, avoid `RandomNumberGenerator.GetBytes(int)` and `SHA1.HashData` in multi-target test code; use helper wrappers for net48/net472 compatibility.
- Citations: Tests/Opc.Ua.Core.Tests/Stack/Types/UserNameIdentityTokenHandlerTests.cs:340-353; CodeQL run 24633036015 job 72024739764 errors at lines 95,133,164,195,233-235,254

compatibility
- Fact: For multi-target builds, avoid direct `CryptographicOperations.ZeroMemory`/`SHA1.HashData` in Opc.Ua.Core codepaths that compile for net48/net472; use framework-compatible helpers.
- Citations: Stack/Opc.Ua.Core/Stack/Types/UserNameIdentityTokenHandler.cs:714-736; GitHub Actions CodeQL run 24628596918 job 72013904073 compile errors for net48/net472

build commands
- Fact: Types tests can be validated with: dotnet test Tests/Opc.Ua.Types.Tests/Opc.Ua.Types.Tests.csproj -c Release -f net10.0
- Citations: User input task run; verified in this session with passing run (7485 tests) in /home/runner/work/UA-.NETStandard/UA-.NETStandard

xml parser arrays
- Fact: XmlParser.ReadEnumeratedArray(string) now throws BadDecodingError when the field exists but has no child elements, including empty or text-only content.
- Citations: Stack/Opc.Ua.Types/Encoders/XmlParser.cs:2035-2042, Tests/Opc.Ua.Types.Tests/Encoders/XmlParserTests.cs:1548-1608

constants
- Fact: Use generated DataTypeIds.RsaEncryptedSecret instead of hardcoded 17545 in UserNameIdentityTokenHandler and tests.
- Citations: Stack/Opc.Ua.Core/Stack/Types/UserNameIdentityTokenHandler.cs:259,469; Tests/Opc.Ua.Core.Tests/Stack/Types/UserNameIdentityTokenHandlerTests.cs:231

constants
- Fact: Use DataTypeIds.RsaEncryptedSecret for RSAEncryptedSecret type checks/encoding instead of local numeric IDs.
- Citations: Stack/Opc.Ua.Core/Stack/Types/UserNameIdentityTokenHandler.cs:303, Stack/Opc.Ua.Core/Stack/Types/UserNameIdentityTokenHandler.cs:267, Tests/Opc.Ua.Core.Tests/Stack/Types/UserNameIdentityTokenHandlerTests.cs:222

xml parser peek
- Fact: XmlParser.Peek(XmlNodeType.Element) falls back to current context element when no unconsumed child exists; array code must check MoveToElement(null) to confirm child presence.
- Citations: Stack/Opc.Ua.Types/Encoders/XmlParser.cs:231-238, Stack/Opc.Ua.Types/Encoders/XmlParser.cs:2702-2715, Stack/Opc.Ua.Types/Encoders/XmlParser.cs:2007-2016

encoder behavior
- Fact: XmlDecoder and XmlParser ReadEnumeratedArray(string) must throw BadDecodingError when Peek(XmlNodeType.Element) returns null (no decodable child elements).
- Citations: Stack/Opc.Ua.Types/Encoders/XmlDecoder.cs:2001-2012, Stack/Opc.Ua.Types/Encoders/XmlParser.cs:2001-2018, Tests/Opc.Ua.Types.Tests/Encoders/XmlDecoderTests.cs:2143-2164, Tests/Opc.Ua.Types.Tests/Encoders/XmlParserTests.cs:1547-1568

Be sure to consider these stored facts carefully. Consider whether any are relevant to your current task. If they are, verify their current applicability before using them to inform your work.

If you come across a memory that you're able to verify and that you find useful, you should use the store_memory tool to store the same fact again. Only recent memories are retained, so storing the fact again will cause it to be retained longer.

```
If you come across a fact that's incorrect or outdated, you
should use the store_memory tool to store a new fact that
reflects the current reality.
Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address
them. Ignore comments that are not actionable or directed at
you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you
can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment
and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>




A **CI/Build Failure** has been mentioned in the problem
statement, make sure to follow your instructions for addressing
CI/Build Failures.
```